Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №1

по дисциплине «Серверное программирование»

Тема: проектирование и создание базы данных

Преподаватель _____ _____ _____
подпись, дата | инициалы, фамилия

Студент _____ _____ Mohamed Amini
номер группы, зачетной книжки | подпись, дата | инициалы, фамилия

Красноярск 2026

1 Цель и постановка задачи

Цель работы – спроектировать архитектуру базы данных выбранной предметной области и реализовать базовые таблицы с данными. База данных будет использоваться во всех последующих практических работах курса «Серверное программирование».

Требования задания:

– определить предметную область и оформить архитектуру базы данных (ER- или UML-диаграмма, не менее 5 таблиц);

– создать схему данных: таблицы, типы столбцов, первичные ключи;

– заполнить каждую таблицу не менее чем 10 записями;

– составить запрос CROSS JOIN по всем созданным таблицам.

Критерии приёмки: диаграмма модели данных, DDL-скрипт создания таблиц, скрипт заполнения данными и запрос CROSS JOIN с результатом выполнения. Отчёт содержит диаграмму, SQL-скрипты и результаты выполнения запросов.

2 Стек и запуск

Работа выполнена на СУБД SQLite, которая подключается встроенным модулем node:sqlite платформы Node.js 24; программа на TypeScript выполняется без шага сборки. Альтернативный стек допускается разделом C задания и согласован с преподавателем.

Состав проекта: каталог sql содержит три SQL-скрипта (схема, данные, запрос CROSS JOIN), файл src/run.ts создаёт файл базы данных velograd.db, выполняет первые два скрипта, выводит число строк в каждой таблице и выполняет запросы третьего скрипта. Запуск: npm start.

3 Предметная область и модель данных

Предметная область – сеть городского велопроката «ВелоГрад»: районы города, станции проката в районах, велосипеды, клиенты, тарифы, поездки и

платежи. Модель состоит из семи таблиц, сведения о которых приведены в таблице 1; диаграмма – на рисунке 1.

Таблица 1 – Таблицы базы данных

Таблица	Содержание	Внешние ключи	Строк
district	район города	–	10
station	станция проката	district_id → district	12
bicycle	велосипед	home_station_id → station	14
client	клиент	–	12
tariff	тариф	–	10
rental	поездка	client_id, bicycle_id, tariff_id, start_station_id, end_station_id	16
payment	платёж за поездку	rental_id → rental (уникальный)	12

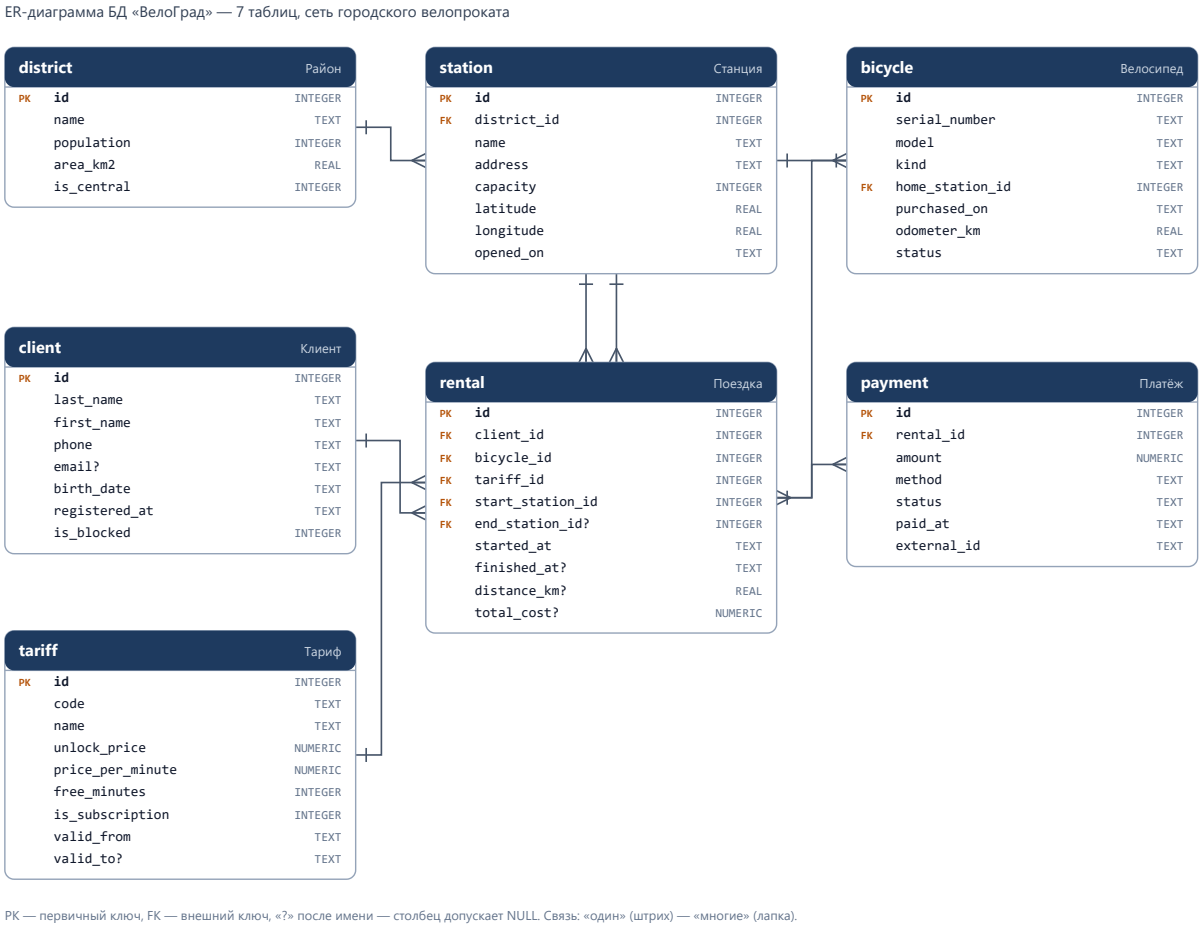


Рисунок 1 – ER-диаграмма базы данных «ВелоГрад»

Связи модели: район – станция, станция – велосипед, клиент – поездка, велосипед – поездка, тариф – поездка – «один ко многим»; станция связана с

поездкой дважды (станция начала и станция конца поездки); поездка – платёж – «один к одному», для чего столбец payment.rental_id объявлен уникальным. Каждая таблица имеет первичный ключ id.

4 Схема данных

Первичные ключи объявлены как INTEGER PRIMARY KEY AUTOINCREMENT. Даты и время хранятся в столбцах TEXT с проверкой формата функциями date() и datetime(), логические значения – в INTEGER со значениями 0 и 1. Корректность данных обеспечивают ограничения NOT NULL, UNIQUE и CHECK; внешние ключи заданы с действием ON DELETE RESTRICT (для платежа – CASCADE). Для внешних ключей и часто используемых столбцов созданы индексы. Скрипт sql/01_schema.sql:

```
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS payment;
DROP TABLE IF EXISTS rental;
DROP TABLE IF EXISTS tariff;
DROP TABLE IF EXISTS client;
DROP TABLE IF EXISTS bicycle;
DROP TABLE IF EXISTS station;
DROP TABLE IF EXISTS district;

CREATE TABLE district (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT    NOT NULL UNIQUE,
    population  INTEGER NOT NULL CHECK (population > 0),
    area_km2    REAL    NOT NULL CHECK (area_km2 > 0),
    is_central  INTEGER NOT NULL DEFAULT 0 CHECK (is_central IN (0, 1))
);

CREATE TABLE station (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    district_id  INTEGER NOT NULL,
    name         TEXT    NOT NULL,
    address      TEXT    NOT NULL,
    capacity     INTEGER NOT NULL CHECK (capacity BETWEEN 4 AND 60),
    latitude     REAL    NOT NULL CHECK (latitude  BETWEEN -90  AND 90),
    longitude    REAL    NOT NULL CHECK (longitude BETWEEN -180 AND 180),
    opened_on    TEXT    NOT NULL CHECK (opened_on IS date(opened_on)),

    CONSTRAINT uq_station_name_in_district UNIQUE (district_id, name),
    CONSTRAINT fk_station_district FOREIGN KEY (district_id)
        REFERENCES district (id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE bicycle (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    serial_number   TEXT    NOT NULL UNIQUE,
    model           TEXT    NOT NULL,
    kind            TEXT    NOT NULL CHECK (kind IN ('city', 'electric', 'cargo', 'teen')),
    home_station_id INTEGER NOT NULL,
    purchased_on    TEXT    NOT NULL CHECK (purchased_on IS date(purchased_on)),
    odometer_km     REAL    NOT NULL DEFAULT 0 CHECK (odometer_km >= 0),
    status          TEXT    NOT NULL DEFAULT 'free'
                            CHECK (status IN ('free', 'rented', 'service', 'written_off')),

    CONSTRAINT fk_bicycle_station FOREIGN KEY (home_station_id)
```

```
        REFERENCES station (id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE client (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    last_name     TEXT    NOT NULL,
    first_name    TEXT    NOT NULL,
    phone         TEXT    NOT NULL UNIQUE CHECK (length(phone) = 12 AND phone LIKE '+7%'),
    email         TEXT    NULL UNIQUE CHECK (email IS NULL OR email LIKE '%_@_%._%'),
    birth_date    TEXT    NOT NULL CHECK (birth_date IS date(birth_date)),
    registered_at TEXT    NOT NULL DEFAULT (datetime('now')),
    is_blocked    INTEGER NOT NULL DEFAULT 0 CHECK (is_blocked IN (0, 1))
);

CREATE TABLE tariff (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    code             TEXT    NOT NULL UNIQUE,
    name             TEXT    NOT NULL,
    unlock_price     NUMERIC NOT NULL CHECK (unlock_price >= 0),
    price_per_minute NUMERIC NOT NULL CHECK (price_per_minute >= 0),
    free_minutes     INTEGER NOT NULL DEFAULT 0 CHECK (free_minutes >= 0),
    is_subscription  INTEGER NOT NULL DEFAULT 0 CHECK (is_subscription IN (0, 1)),
    valid_from       TEXT    NOT NULL CHECK (valid_from IS date(valid_from)),
    valid_to         TEXT    NULL     CHECK (valid_to IS NULL OR valid_to > valid_from)
);

CREATE TABLE rental (
    id               INTEGER PRIMARY KEY AUTOINCREMENT,
    client_id        INTEGER NOT NULL,
    bicycle_id       INTEGER NOT NULL,
    tariff_id        INTEGER NOT NULL,
    start_station_id INTEGER NOT NULL,
    end_station_id   INTEGER NULL,
    started_at       TEXT    NOT NULL CHECK (started_at IS datetime(started_at)),
    finished_at      TEXT    NULL     CHECK (finished_at IS NULL OR finished_at IS datetime(finished_at)),
    distance_km      REAL    NULL     CHECK (distance_km IS NULL OR distance_km >= 0),
    total_cost       NUMERIC NULL     CHECK (total_cost  IS NULL OR total_cost  >= 0),

    CONSTRAINT ck_rental_finished CHECK (finished_at IS NULL OR finished_at > started_at),

    CONSTRAINT ck_rental_complete CHECK (
        (finished_at IS     NULL AND end_station_id IS     NULL AND total_cost IS     NULL) OR
        (finished_at IS NOT NULL AND end_station_id IS NOT NULL AND total_cost IS NOT NULL)
    ),

    CONSTRAINT fk_rental_client  FOREIGN KEY (client_id)        REFERENCES client  (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_bicycle FOREIGN KEY (bicycle_id)       REFERENCES bicycle (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_tariff  FOREIGN KEY (tariff_id)        REFERENCES tariff  (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_start   FOREIGN KEY (start_station_id) REFERENCES station (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_rental_end     FOREIGN KEY (end_station_id)   REFERENCES station (id) ON DELETE RESTRICT
);

CREATE TABLE payment (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    rental_id   INTEGER NOT NULL UNIQUE,
    amount      NUMERIC NOT NULL CHECK (amount > 0),
    method      TEXT    NOT NULL CHECK (method IN ('card', 'sbp', 'wallet', 'cash')),
    status      TEXT    NOT NULL DEFAULT 'pending'
                        CHECK (status IN ('pending', 'succeeded', 'failed', 'refunded')),
    paid_at     TEXT    NOT NULL CHECK (paid_at IS datetime(paid_at)),
    external_id TEXT    NOT NULL UNIQUE,

    CONSTRAINT fk_payment_rental FOREIGN KEY (rental_id)
        REFERENCES rental (id) ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE INDEX idx_station_district  ON station (district_id);
CREATE INDEX idx_bicycle_station   ON bicycle (home_station_id);
CREATE INDEX idx_bicycle_status    ON bicycle (status);
CREATE INDEX idx_rental_client     ON rental  (client_id);
CREATE INDEX idx_rental_bicycle    ON rental  (bicycle_id);
```

```
CREATE INDEX idx_rental_tariff     ON rental  (tariff_id);
CREATE INDEX idx_rental_start      ON rental  (start_station_id);
CREATE INDEX idx_rental_end        ON rental  (end_station_id);
CREATE INDEX idx_rental_started_at ON rental  (started_at);
```

5 Заполнение таблиц данными

Скрипт sql/02_seed.sql добавляет осмысленные данные: в каждой таблице не менее 10 записей. Первые строки скрипта:

```
INSERT INTO district (id, name, population, area_km2, is_central) VALUES
    ( 1, 'Центральный',   128400, 12.4, 1),
    ( 2, 'Приморский',    215600, 34.8, 0),
    ( 3, 'Заречный',       96300, 21.2, 0),
    ( 4, 'Октябрьский',   174900, 27.5, 0),
    ( 5, 'Ленинский',     203100, 31.9, 0),
    ( 6, 'Советский',     142700, 19.6, 0),
    ( 7, 'Кировский',      88500, 15.3, 0),
    ( 8, 'Московский',    231800, 42.1, 0),
    ( 9, 'Красногорский',  76400, 24.7, 0),
    (10, 'Северный',      119200, 38.4, 0);

INSERT INTO station (id, district_id, name, address, capacity, latitude, longitude, opened_on)
VALUES
    ( 1,  1, 'Площадь Революции',  'ул. Тверская, 1',            24, 55.7558, 37.6176, '2021-05-
14'),
    ( 2,  1, 'Городской сад',      'Садовая ул., 12',            18, 55.7601, 37.6215, '2021-05-
14'),
    ( 3,  2, 'Морской вокзал',     'Портовая ул., 3',            30, 55.7712, 37.5904, '2021-07-
02'),
    ( 4,  2, 'Приморский бульвар', 'Приморский б-р, 7',          20, 55.7745, 37.5842, '2022-04-
18'),
```

Число строк в таблицах после выполнения скрипта показано на рисунке 2.

```
PS ...\pz1-database-design> npm start
district: 10 rows
station: 12 rows
bicycle: 14 rows
client: 12 rows
tariff: 10 rows
rental: 16 rows
payment: 12 rows
```

Рисунок 2 – Число строк в таблицах

6 Запрос CROSS JOIN

Декартово произведение соединяет каждую строку одной таблицы с каждой строкой другой, поэтому число строк результата равно произведению числа строк всех таблиц: 10 × 12 × 14 × 12 × 10 × 16 × 12 = 38 707 200. Скрипт

sql/03_cross_join.sql содержит три запроса: ожидаемое число строк, фактическое число строк и первые пять строк результата.

```
SELECT
    (SELECT COUNT(*) FROM district) AS district,
    (SELECT COUNT(*) FROM station)  AS station,
    (SELECT COUNT(*) FROM bicycle)  AS bicycle,
    (SELECT COUNT(*) FROM client)   AS client,
    (SELECT COUNT(*) FROM tariff)   AS tariff,
    (SELECT COUNT(*) FROM rental)   AS rental,
    (SELECT COUNT(*) FROM payment)  AS payment,
    (SELECT COUNT(*) FROM district)
  * (SELECT COUNT(*) FROM station)
  * (SELECT COUNT(*) FROM bicycle)
  * (SELECT COUNT(*) FROM client)
  * (SELECT COUNT(*) FROM tariff)
  * (SELECT COUNT(*) FROM rental)
  * (SELECT COUNT(*) FROM payment) AS expected_rows;

SELECT COUNT(*) AS actual_rows
FROM district
CROSS JOIN station
CROSS JOIN bicycle
CROSS JOIN client
CROSS JOIN tariff
CROSS JOIN rental
CROSS JOIN payment;

SELECT *
FROM district
CROSS JOIN station
CROSS JOIN bicycle
CROSS JOIN client
CROSS JOIN tariff
CROSS JOIN rental
CROSS JOIN payment
LIMIT 5;
```

Ожидаемое число строк, вычисленное как произведение, и число строк фактического соединения показаны на рисунках 3 и 4.

```
PS ...\pz1-database-design> npm start
district | station | bicycle | client | tariff | rental | payment | expected_rows
10 | 12 | 14 | 12 | 10 | 16 | 12 | 38707200
```

Рисунок 3 – Ожидаемое число строк CROSS JOIN

```
PS ...\pz1-database-design> npm start
actual_rows
38707200
```

Рисунок 4 – Фактическое число строк CROSS JOIN

Фактическое число строк совпало с произведением. Результат содержит 55 столбцов (все столбцы семи таблиц); первые строки приведены на рисунке 5

(строки сокращены по ширине рисунка, различаются только последние столбцы – в соединении быстрее всего меняется платёж).

```
PS ...\pz1-database-design> npm start
id | name | population | area_km2 | is_central | id | district_id | name | address | capacity | latitude | l
…
1 | Центральный | 128400 | 12.4 | 1 | 1 | 1 | Площадь Революции | ул. Тверская, 1 | 24 | 55.7558 | 37.6176 |
…
1 | Центральный | 128400 | 12.4 | 1 | 1 | 1 | Площадь Революции | ул. Тверская, 1 | 24 | 55.7558 | 37.6176 |
…
```

Рисунок 5 – Первые строки результата CROSS JOIN

7 Выводы

1 Спроектирована модель данных предметной области «ВелоГрад» из семи таблиц; связи «один ко многим» и «один к одному» отражены в диаграмме и реализованы внешними ключами.

2 Схема создана скриптом с типами столбцов, первичными ключами, ограничениями целостности и индексами; каждая таблица заполнена данными (от 10 до 16 записей).

3 Запрос CROSS JOIN по всем таблицам вернул 38 707 200 строк, что равно произведению числа строк таблиц; это показывает, как быстро растёт декартово произведение и почему соединения без условия на практике применяют только для перебора комбинаций небольших справочников.